

Arquitectura e Ingeniería de
Computadores

Grado de Ingeniería del Software

**TEMA 7. INTRODUCCIÓN A LAS
ARQUITECTURAS AVANZADAS DE
COMPUTADORES**

Ángel Serrano

CONTENIDOS

- Introducción
- Paralelismo a nivel de instrucción
 - Segmentación de instrucciones
 - Riesgos de datos RAW
 - Adelantamientos
 - Procesadores multifuncionales
 - Riesgos de datos WAW
 - Planificación dinámica de instrucciones
 - Riesgos de control (saltos condicionales)
 - Procesadores superescalares
- Paralelismo a nivel de hilo
- Paralelismo a nivel de procesador
 - Procesadores multinúcleo
 - Multiprocesadores simétricos
 - Acceso no uniforme a memoria
 - *Clusters*
- Paralelismo de datos
 - Procesadores vectoriales
 - Unidad de procesamiento gráfico (GPU)
- Despedida

INTRODUCCIÓN

- Durante toda la historia de la Arquitectura de Computadores, se ha intentado compatibilizar un procesador cada vez más rápido con unos componentes que no lo son tanto.
- Algunas de las técnicas utilizadas son:
 - Memoria caché.
 - Memoria virtual.
 - Buses serie frente a paralelos.
- En este tema hablaremos de **paralelismo**:
 - A nivel de instrucción
 - A nivel de hilo
 - A nivel de procesador
 - A nivel de datos

3

- En esta asignatura hemos estudiado el modelo de von Neumann, en el cual el procesador o CPU es siempre la unidad funcional más rápida e importante.
 - Todas las técnicas explicadas tenían como objetivo maximizar el rendimiento del computador, haciendo compatible el uso a la vez de una CPU muy rápida junto con otros componentes (memoria, periféricos, buses) mucho más lentos.
 - Ejemplos:
 - Organizar la memoria con una jerarquía.
 - Uso de memoria caché de varios niveles.
 - “Engañar” a procesador para que crea tener más memoria de la que realmente tiene y así tener varios programas en memoria a la vez.
 - Memoria virtual.
 - Pasar de buses síncronos paralelos a buses asíncronos serie de alta capacidad (como PCIe).
 - En este tema haremos un breve repaso de técnicas avanzadas de Arquitectura de Computadores, cotidianas hoy en día, para mejorar aún más el rendimiento del computador.
 - Se basan en el paralelismo a nivel de instrucción.
 - Se dispone de un único procesador que permite la ejecución simultánea de varias instrucciones del programa.
 - Para ello disponen de múltiples unidades funcionales (ALUs).
 - Otras se basan en el paralelismo a nivel de hilo.
 - Un único procesador permite ejecutar simultáneamente varios hilos (secuencias de instrucciones) del mismo programa.
 - Cuando disponemos de varios procesadores, trabajando independientemente, hablamos de paralelismo a nivel de procesador.
 - Puede ser un procesador con varios núcleos o varios procesadores separados físicamente.
 - La configuración concreta complica los accesos a la memoria principal, algo que habrá que tener en cuenta.
 - También se complica el manejo de la caché (problemas de coherencia).
 - Otra posibilidad es trabajar con ordenadores completos totalmente independientes, pero coordinados entre sí para una tarea común (*cluster*).
 - Por último, hablaremos de las técnicas que explotan el paralelismo a nivel de datos:
 - Es decir, realizan el procesamiento simultáneo de muchos datos.
 - Un ejemplo sería acceder a múltiples componentes de un vector.

INTRODUCCIÓN

TAXONOMÍA DE FLYNN:

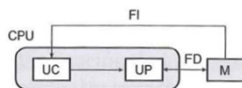
	<i>Single data</i>	<i>Multiple data</i>
<i>Single instruction</i>	SISD	SIMD
<i>Multiple instruction</i>	MISD	MIMD

4

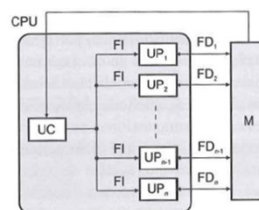
- La taxonomía (clasificación) de Flynn fue propuesta por Michael J. Flynn en 1972.
 - Clasifica los computadores en función de si tienen uno o múltiples flujos de instrucción, y de si tienen uno o múltiples flujos de datos.
 - En cuanto a los flujos de instrucción:
 - Se tiene un único flujo de instrucción (SI, *single instruction*) cuando un único procesador va ejecutando las instrucciones.
 - En caso de haber varios procesadores capaces de ejecutar diferentes instrucciones de manera independiente (MI, *multiple instruction*), estaríamos hablando de múltiples flujos de instrucción.
 - En cuanto a los flujos de datos:
 - Si vamos tratando los datos de uno en uno, es decir, de manera escalar (SD, *single data*), tendríamos un único flujo de datos.
 - Si, por el contrario, podemos trabajar con varios datos a la vez, es decir, de manera vectorial (MD, *multiple data*), tendríamos múltiples flujos de datos.

INTRODUCCIÓN

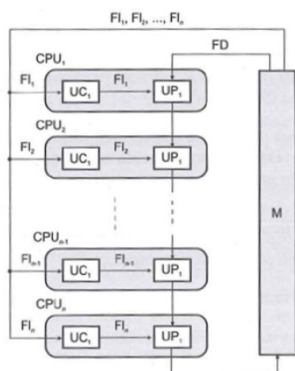
SISD



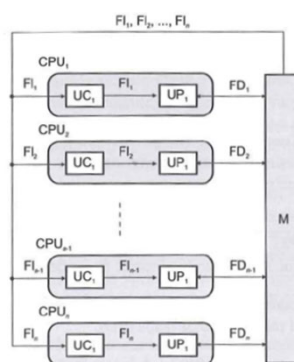
SIMD



MISD



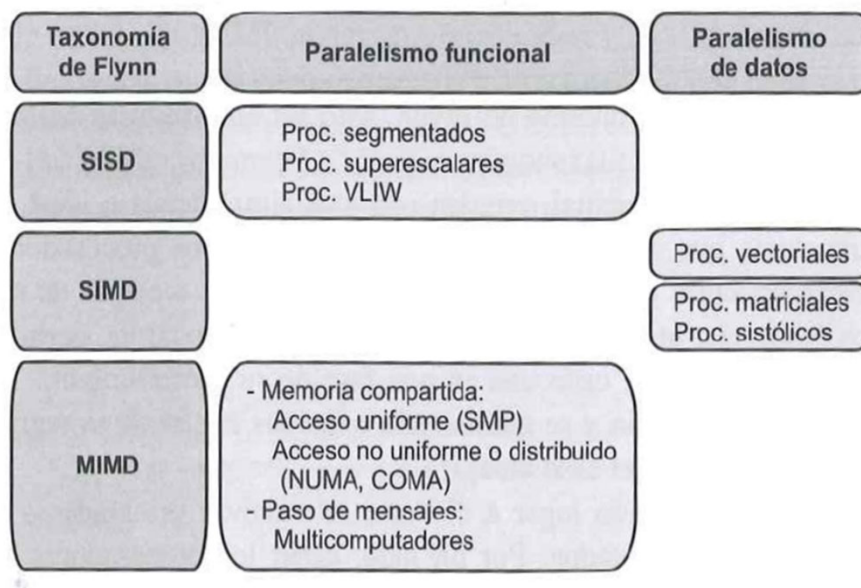
MIMD



- **SISD (single instruction, single data):**
 - Es el modelo de von Neumann normal (un flujo de instrucciones y un flujo de datos).
 - Una única unidad de control recibe las instrucciones, las decodifica y controla la unidad de proceso de datos.
 - Tipos: computadores monoprocesador.
 - Ejemplo: Intel Pentium 4.
- **SIMD (single instruction, multiple data):**
 - Las distintas unidades de proceso de datos están controladas por una única unidad de control, que ordena a todas que realicen la misma operación simultáneamente, cada una manejando sus propios datos.
 - Así, por ejemplo, la suma de las 64 componentes de un vector podría realizarse en un único ciclo de reloj si dispusiéramos de 64 ALUs.
 - Una única instrucción (según el contador de programa PC) activaría las ALUs, pero cada una de ellas dispondría de sus propios registros y accedería a datos diferentes.
 - Desde el punto de vista del programador, SIMD es equivalente al modelo SISD.
 - SIMD funciona mejor si existe paralelismo a nivel de datos, es decir, los datos están estructurados de tal manera que es posible realizar la misma operación en componentes separadas.
 - Un ejemplo es el de suma de matrices, donde la suma se realiza elemento a elemento y no depende de las otras componentes.
 - Es por ello que las arquitecturas vectoriales son:
 - Más eficientes ejecutando bucles *for*, donde en cada iteración se accede a un conjunto independiente de los datos.
 - Muy ineficientes ejecutando bucles *switch-case*.
 - Tipos: procesadores matriciales y vectoriales.
 - Ejemplo: conjunto de instrucciones SSE (*Streaming SIMD Extensions*) y AVX (*Advanced Vector Extensions*) de la arquitectura x86.
- **MISD (multiple instruction, single data):**
 - Varias unidades de control decodifican instrucciones correspondientes a diferentes programas, pero los datos pasan de una unidad de proceso a otra en un flujo único.
 - Se pueden simular mediante computadores MIMD, con lo que no existen computadores MISD específicos.
- **MIMD (multiple instruction, multiple data):**
 - Varias unidades de control decodifican instrucciones correspondientes a diferentes programas, cada uno de los cuales tiene su propio flujo de datos.
 - Tipos: multiprocesadores y multicomputadores.
 - Ejemplo: Intel Core i7.

Imagen tomada de: Sebastián Dormido Canto et al., Ingeniería de Computadores II, Editorial Sanz y Torres, 2011, página 8.

INTRODUCCIÓN



6

- Esta imagen resume la taxonomía de Flynn con las distintas arquitecturas según su paralelismo funcional o de datos.
 - El paralelismo funcional se refiere a que se dispone de múltiples componentes del hardware (unidades funcionales) que permiten la ejecución de varias instrucciones a la vez.
 - Caso SISD:
 - Las arquitecturas segmentadas, superescalares y supersegmentadas son diversas formas de paralelismo a nivel de instrucción (ILP, *instruction-level parallelism*).
 - En todas ellas hay un único procesador con un único flujo de instrucciones y de datos (SISD).
 - En el caso segmentado, el procesador está ejecutando varias instrucciones en distintas fases de ejecución.
 - En el caso superescalar, el procesador es capaz de leer y ejecutar varias instrucciones a la vez, ya que dispone de unidades de procesamiento múltiple.
 - Los procesadores VLIW (*very long instruction word*) disponen de instrucciones máquina muy largas en las que el compilador empaqueta instrucciones que pueden ejecutarse simultáneamente.
 - Caso MIMD:
 - Los procesadores MIMD disponen de múltiples flujos de instrucción, independientes entre sí, que acceden a su vez a múltiples flujos de datos.
 - Este es el caso de los multiprocesadores (computadores basados en varios procesadores trabajando en paralelo) o de los procesadores multinúcleo (disponen de varios procesadores grabados en el chip, en la misma oblea de silicio).
 - Les es necesario algún mecanismo de comunicación y coordinación entre los distintos procesadores, como puede ser el uso de:
 - Memoria compartida:
 - Con tiempo de acceso a la memoria uniforme o SMP = *Symmetric Multiprocessor*.
 - Con tiempo de acceso no uniforme o NUMA = *Non-Uniform Memory Access*, COMA = *Cache-Only Memory Access*, entre otros).
 - O bien mediante paso de mensajes (caso de los multicomputadores).
 - El paralelismo de datos (SIMD) se refiere a que los datos intrínsecos con los que trabajamos permiten la realización de cálculos múltiples de manera simultánea.
 - Los procesadores vectoriales o los matriciales son ejemplos de arquitecturas SIMD, ya que aprovechan el paralelismo de los datos.
 - El procesador ejecuta un único flujo de instrucciones que se aplica a muchos datos almacenados como componentes de vectores o matrices.
 - Es decir, se aplica la misma operación (=instrucción) a cada una de las componentes.

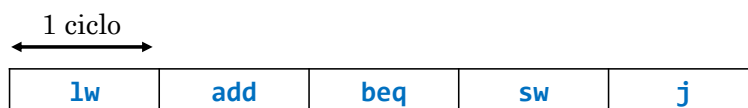
Imagen tomada de: Sebastián Dormido Canto et al., Ingeniería de Computadores II, Editorial Sanz y Torres, 2011, página 12.

CONTENIDOS

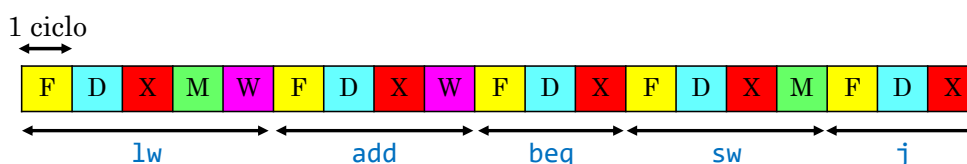
- Introducción
- **Paralelismo a nivel de instrucción**
 - Segmentación de instrucciones
 - Riesgos de datos RAW
 - Adelantamientos
 - Procesadores multifuncionales
 - Riesgos de datos WAW
 - Planificación dinámica de instrucciones
 - Riesgos de control (saltos condicionales)
 - Procesadores superescalares
- Paralelismo a nivel de hilo
- Paralelismo a nivel de procesador
 - Procesadores multinúcleo
 - Multiprocesadores simétricos
 - Acceso no uniforme a memoria
 - *Clusters*
- Paralelismo de datos
 - Procesadores vectoriales
 - Unidad de procesamiento gráfico (GPU)
- Despedida

HASTA AHORA

○ MIPS unicycle (CPI = 1):



○ MIPS multiciclo (CPI > 1):



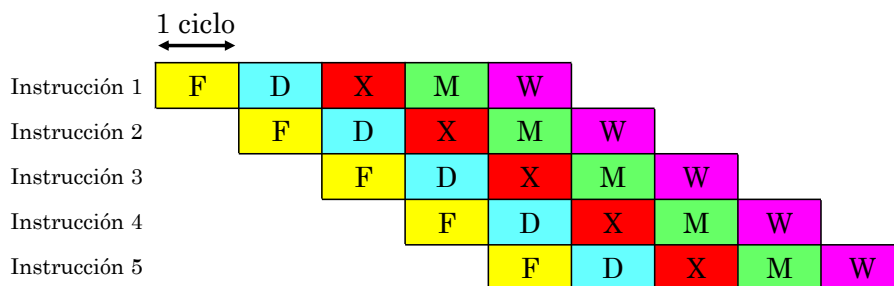
○ CPI = Ciclos por instrucción.

8

- Durante el curso hemos estudiado dos posibles implementaciones del camino de datos de MIPS:
 - Uniciclo (o monociclo):
 - Todas las instrucciones tardan 1 ciclo de reloj en completarse.
 - Ese decir, CPI = 1.
 - En un momento dado hay solo 1 instrucción ejecutándose.
 - El ciclo de reloj debe ser lo suficientemente largo para permitir que se complete la instrucción más lenta (la que hace más pasos).
 - Hay instrucciones que podrían ejecutarse en menos tiempo, como beq o j, pero no es factible usar un periodo de reloj variable.
 - No se puede usar el mismo componente más de una vez por ciclo.
 - Por eso, tenemos memorias separadas para datos e instrucciones.
 - Multiciclo:
 - En un momento dado sigue habiendo solo 1 instrucción ejecutándose.
 - Ahora las instrucciones se dividen en etapas o fases.
 - Cada instrucción se ejecuta en un número entero de etapas, cada una de las cuales se ejecuta en 1 ciclo.
 - La duración del ciclo de reloj es mucho más corta que en el caso uniciclo.
 - Se ajusta ahora a la etapa más larga, no a la instrucción más complicada.
 - Las etapas son:
 - Búsqueda de la instrucción (*Instruction Fetch*, en la figura: F).
 - Decodificación de la instrucción (*Instruction Decode*, en la figura: D).
 - Ejecución (*Execution*, en la figura: X).
 - Acceso a memoria (*Memory*, en la figura: M).
 - Escritura en el banco de registros (*Write Back*, en la figura: W).
 - Se han usado estas abreviaturas y no las vistas en el tema 3 (IF, ID, EX, MEM y WB) por cuestiones de claridad en la figura.
 - El número de etapas que necesita cada instrucción depende de la instrucción y se ajusta mejor a cuántas operaciones tiene que realizar:
 - beq y j: 3 ciclos.
 - Aritmético-lógicas de formato R y sw: 4 ciclos.
 - lw: 5 ciclos.
 - En este caso CPI > 1.
 - El valor exacto depende de la frecuencia de aparición de cada tipo de instrucción (media ponderada).
 - En el ejemplo de la figura se utilizan 19 ciclos para 5 instrucciones: $CPI = 19/5 = 3,8$.
 - Un mismo componente se puede usar varias veces en una instrucción, siempre y cuando esto ocurra en ciclos diferentes.
 - Por eso, tenemos una memoria unificada para datos e instrucciones.

SEGMENTACIÓN DE INSTRUCCIONES

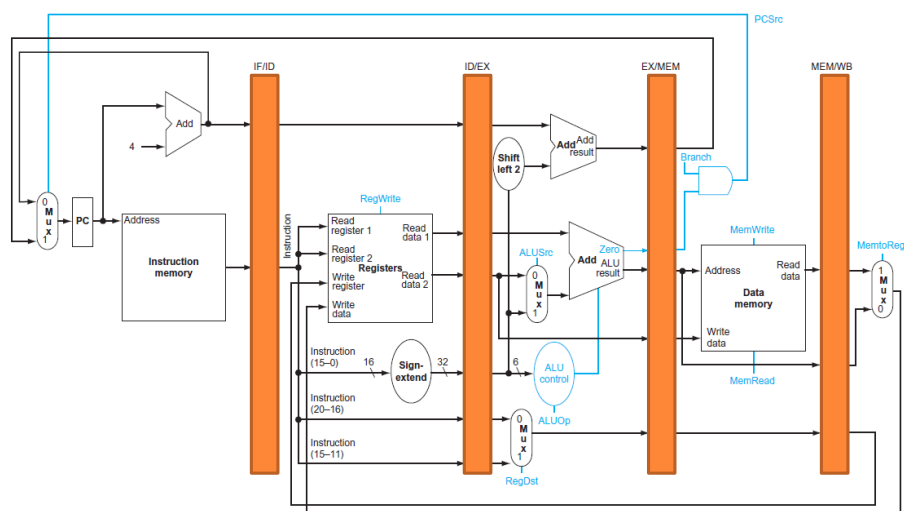
- Intenta conseguir $CPI = 1$ (como en un ciclo), pero con un ciclo de reloj corto (como multiciclo).
- Cadena de montaje de una fábrica.
- Con 5 etapas de ejecución, se puede solapar la ejecución de 5 instrucciones, cada una en una etapa diferente.



9

- Segmentación de instrucciones (*pipelining*):
 - Es una técnica similar a la utilizada en las cadenas de montaje de una fábrica.
 - Su objetivo es:
 - Conseguir que se termine por completo 1 instrucción en cada ciclo ($CPI = 1$).
 - Similar a lo que hace la implementación un ciclo.
 - Pero permite instrucciones de varios ciclos.
 - Como en multiciclo.
 - En el MIPS segmentado, todas las instrucciones necesitan 5 ciclos.
 - En algunas instrucciones (beq, j, aritmético-lógicas de formato R y sw), hay alguna etapa en la que no se hace nada.
 - Tras las fases iniciales de llenado del cauce o camino de datos, hay ejecutándose a la vez tantas instrucciones como etapas (en MIPS son 5).
 - En la práctica a veces surgen problemas y no se llega a este valor máximo.
 - No confundir la “segmentación de instrucciones” con la “memoria segmentada” (división de la memoria en porciones o segmentos cuyo tamaño se ajusta a su contenido).

SEGMENTACIÓN DE INSTRUCCIONES



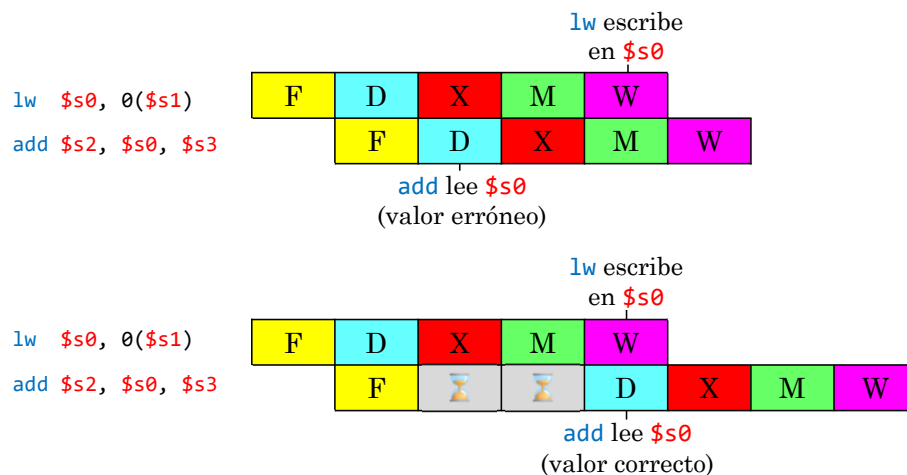
10

- Sin entrar en detalles, se consigue la segmentación modificando el camino de datos:
 - Cada etapa comienza con unos datos que se leen de unos registros, se transforman mediante un cálculo durante el ciclo y se almacena el resultado al final del ciclo.
 - Para ello se añaden grandes registros específicos que separan las partes del camino de datos involucradas en cada etapa.
 - Los valores que se escriben al final del ciclo de una etapa son los que se leen al comienzo del ciclo de la etapa siguiente.
 - Por eso, los registros se llaman:
 - IF/ID
 - ID/EX
 - EX/MEM
 - MEM/WB
 - Dificultades estructurales:
 - Puede coincidir la lectura de una instrucción en la memoria (etapa IF) con la lectura o escritura de un dato en la memoria por parte de otra instrucción (etapa MEM):
 - Se recuperan las memorias separadas de datos e instrucciones (como en un ciclo).
 - También se debe permitir la lectura de dos registros en el banco por parte de una instrucción (etapa ID) con la escritura de un resultado en el banco por otra instrucción (etapa WB).
 - Las escrituras de los registros se hacen en la primera mitad del ciclo.
 - Las lecturas de los registros se hacen en la segunda mitad del ciclo.
 - En el caso de una instrucción beq, la resta de los registros se calcula en la ALU en la etapa EX, pero la activación de la señal FuentePC (PCSrc en la imagen), se produce en la etapa MEM.
 - En el apartado sobre riesgos de control, intentaremos arreglarlo (ver más adelante).

Imagen tomada del libro Patterson & Hennessy, Computer Organization and Design, 5.ª edición, 2014, pág. 301.

RIESGOS DE DATOS RAW (*READ AFTER WRITE*)

- Si se lee un registro en el que debemos escribir previamente, surge un riesgo *Read After Write*.
 - Puede ser necesario introducir ciclos de parada (de espera).

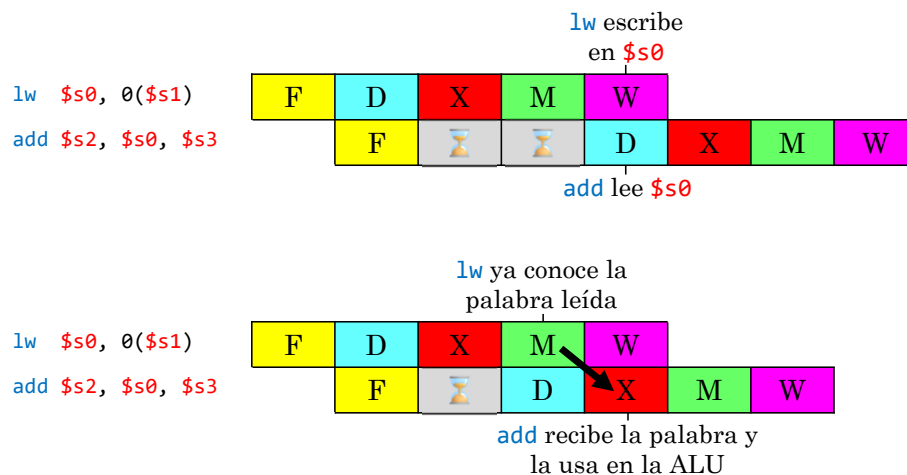


11

- Si las instrucciones tienen dependencias entre sí, pueden surgir problemas (riesgos).
- En el ejemplo que se muestra:
 - Problema:
 - La instrucción de carga escribe el valor leído en memoria en el registro \$s0 en el ciclo WB.
 - Sin embargo, la instrucción de suma necesita leer el valor correcto de \$s0 en el ciclo ID.
 - Si ejecutamos la suma del tirón, estaríamos usando el valor antiguo de \$s0 en la suma, con lo que el resultado sería erróneo.
 - Solución:
 - Introducimos dos ciclos de parada o pausa entre las etapas IF e ID de la instrucción de suma.
 - En la primera mitad del quinto ciclo, la instrucción de carga modifica \$s0.
 - Durante la segunda mitad del quinto ciclo, la instrucción de suma lee \$s0 en el banco (ya será el valor correcto).
- Este tipo de riesgos se denominan RAW:
 - Read After Write* o lectura después de escritura.
 - Su detección lo antes posible complica el diseño de la unidad de control, que debe programar los ciclos de parada cuando se necesiten.
- Como una instrucción no puede avanzar de etapa hasta que la anterior no haya avanzado también, la instrucción que va después de add en el ejemplo no podría empezar hasta el ciclo 5 (no en el 3, como habría ocurrido si no hubiera habido paradas).
 - La técnica de paradas resuelve los riesgos RAW, pero aumenta el número total de ciclos, con lo que empeora el CPI.

SEGMENTACIÓN DE INSTRUCCIONES CON ADELANTAMIENTOS

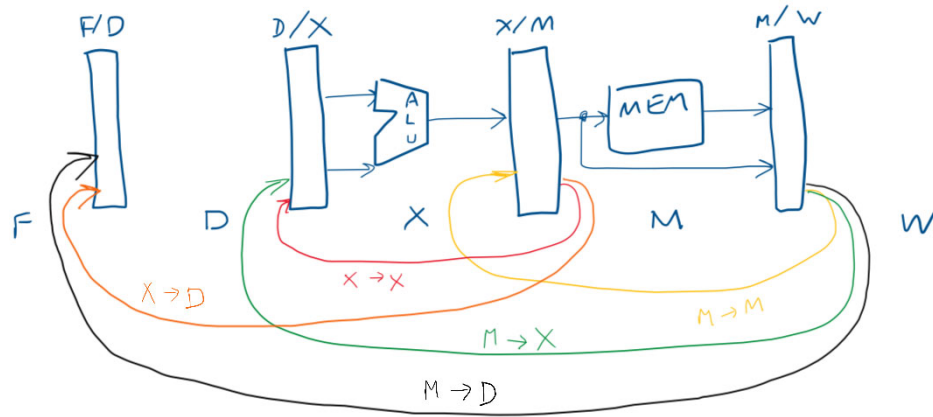
- Consiste en añadir nuevas conexiones que conecten los resultados de una etapa de ejecución con otras etapas de ejecución, para así ahorrar paradas.



12

- La técnica de los adelantamientos (*forwarding*):
 - Permite eliminar algunas paradas (no todas) y, por tanto, ejecutar los programas más rápido.
 - Consiste en modificar el camino de datos para que existan conexiones entre el final de cada etapa de ejecución y el comienzo de otras etapas (obviamente, correspondientes a distintas instrucciones).
 - El adelantamiento puede, en principio, producirse entre cualquier par de etapas de dos instrucciones, siempre que se produzca de un ciclo para el siguiente.
 - Si el dato que se quiere adelantar se va a usar dos o más ciclos después del instante actual, entonces no se puede realizar el adelantamiento.

SEGMENTACIÓN DE INSTRUCCIONES CON ADELANTAMIENTOS



13

- En esta imagen se muestran esquemáticamente las conexiones que serían necesarias para permitir los diferentes adelantamientos.
 - EX → ID (naranja)
 - EX → EX (rojo)
 - MEM → ID (negro)
 - MEM → EX (verde)
 - MEM → MEM (amarillo)

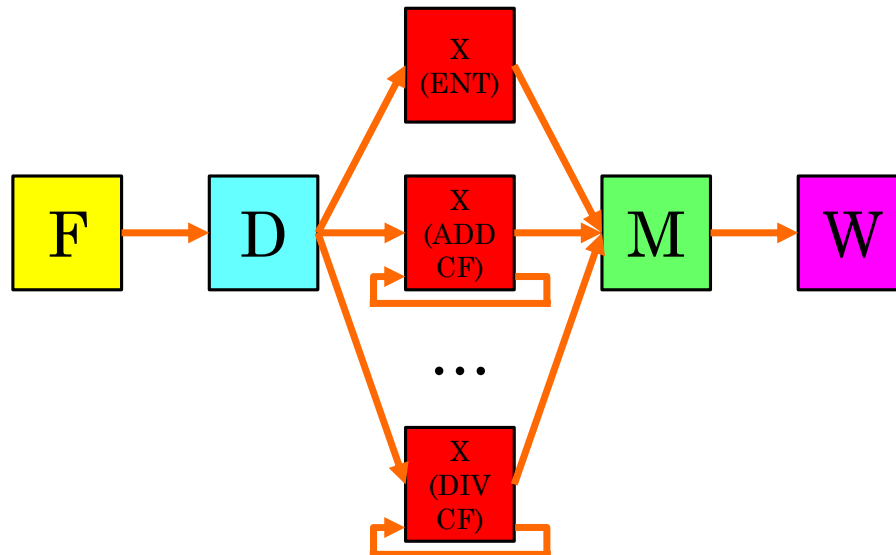
SEGMENTACIÓN EN PROCESADORES MULTIFUNCIONALES

- Hasta ahora, la ALU solo realizaba operaciones enteras.
- Los procesadores actuales pueden realizar también operaciones de coma flotante.
- Los procesadores segmentados multifuncionales disponen de:
 - Una ALU de enteros.
 - Etapa EX = 1 ciclo.
 - Varias ALUs de coma flotante.
 - La duración de la etapa EX ahora dura varios ciclos, según la complejidad de la operación.

14

- Durante todo el curso hemos supuesto por simplicidad que las operaciones aritméticas son de tipo entero.
 - Para ellas, la operación realizada en la ALU durante la etapa EX dura 1 ciclo.
- Si permitimos que además se puedan realizar operaciones aritméticas de coma flotante:
 - Necesitaremos circuitos específicos para estas operaciones.
 - Generalmente los circuitos que realizan sumas y restas en coma flotante son más rápidos que los que realizan multiplicaciones y divisiones (que son más complejas).
 - Ahora la etapa EX puede durar 1 o varios ciclos, en función del tipo de operación.

SEGMENTACIÓN EN PROCESADORES MULTIFUNCIONALES

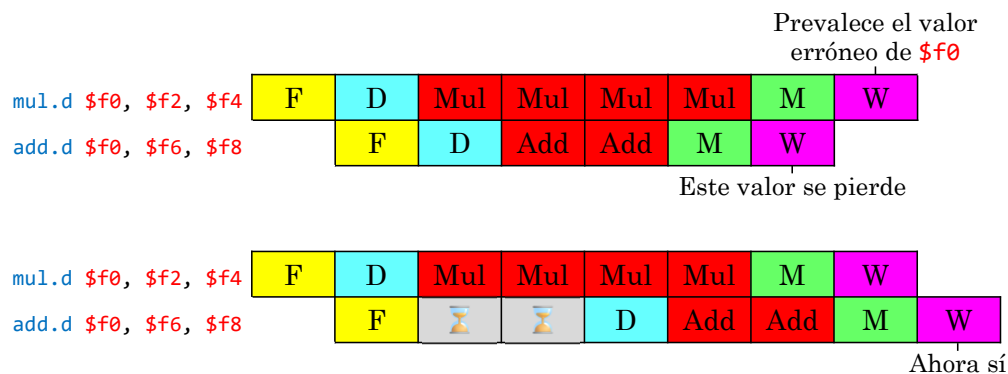


15

- En los procesadores segmentados multifuncionales se modifica el camino de datos para incluir diversos tipos de ALUs que actúan en la etapa EX.
 - Según la operación concreta, la etapa EX durará más o menos ciclos.
 - Si es de coma flotante, se repite la etapa EX tantas veces como ciclos sean necesarios para completar el cálculo.
 - Por eso en el dibujo hay una flecha que apunta de nuevo en la etapa EX para el caso de las operaciones de coma flotante (CF).
 - Para acelerar la ejecución, también se puede optar por segmentar estas ALUs de coma flotante.
 - De esta forma se podrían solapar varias operaciones de coma flotante seguidas, aunque cada una estaría en una subetapa diferente (EX1, EX2, etc.).

RIESGOS DE DATOS WAW (WRITE AFTER WRITE)

- Nuevo riesgo: dos escrituras seguidas (*Write After Write*).
 - El resultado de la segunda escritura es el que debe prevalecer.
 - La unidad de control debe ser capaz de detectar este riesgo y añadir las paradas oportunas.



16

- Anteriormente vimos los riesgos de datos que se producen cuando hay dependencias entre instrucciones del tipo lectura después de escritura (RAW).
- Si permitimos que la duración de las instrucciones sea variable, en función de la operación aritmética realizada, podrían surgir nuevos riesgos:
 - Se trata de la escritura después de escritura.
- En el ejemplo que se muestra:
 - Hemos supuesto que:
 - El circuito que realiza las multiplicaciones de coma flotante necesita 4 ciclos para realizar el cálculo.
 - El que realiza las sumas en coma flotante necesita solo 2 ciclos.
 - Como la suma es más corta que la multiplicación, podría acabar antes que la multiplicación.
 - Pero en ese caso, el valor que prevalecería sería el de la multiplicación, no el de la suma.
 - El riesgo WAW se resuelve añadiendo dos paradas.
 - Necesitamos que la escritura de la suma se produzca después de la escritura de la multiplicación.
 - En el ejemplo se ha optado por incluir las paradas entre las etapas IF e ID de la suma.
 - Esto es porque se presupone que en la etapa de decodificación de la instrucción es cuando se detecta el riesgo WAW.
 - También se podrían haber añadido entre las etapas MEM y WB.

NECESIDAD DE TÉCNICAS DINÁMICAS

- Los riesgos RAW pueden empeorar significativamente el rendimiento.
- Incluso utilizando adelantamientos, no conseguimos eliminar todas las paradas.
- También se puede reordenar el código, que es una técnica de software realizada por el compilador y/o el desarrollador en tiempo de compilación (no de ejecución), lo cual supone una planificación estática de las instrucciones.
- Como consecuencia, el CPI se aleja del valor ideal de 1.

17

- Hasta ahora hemos visto que, en caso de riesgos de datos de tipo RAW (lectura después de escritura), se necesitan añadir varios ciclos de parada que empeoran el rendimiento del procesador.
 - La técnica de los adelantamientos (técnica hardware) permite eliminar alguno de los ciclos de parada, pero no todos.
- También la reordenación de las instrucciones por parte del compilador, o incluso del programador/a (técnicas de software), permite salvar algún ciclo.
 - Pero esta reordenación se realiza de manera estática, es decir, en tiempo de compilación, no en tiempo de ejecución.
- Todos estos inconvenientes nos alejan del valor ideal de $CPI = 1$.
- Se hace necesario incluir técnicas dinámicas (en tiempo de ejecución) para reducir más paradas.

PLANIFICACIÓN DINÁMICA DE INSTRUCCIONES

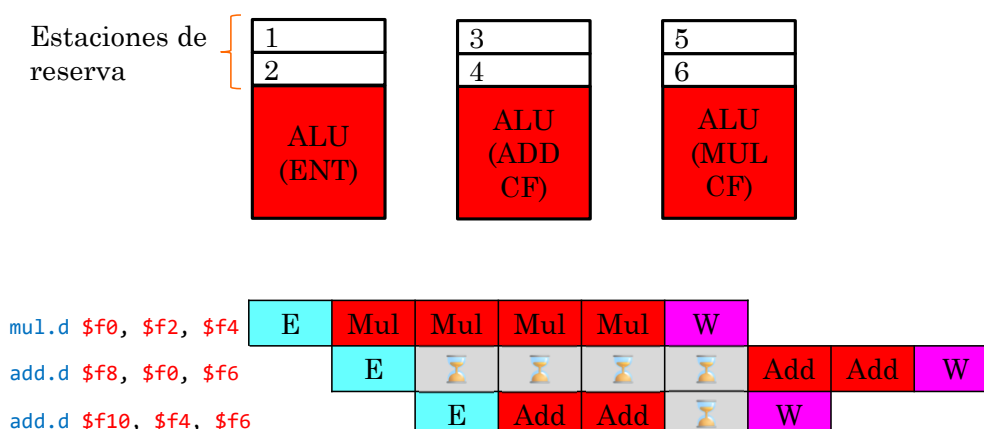
○ Algoritmo de Tomasulo:

- Se añaden una o varias estaciones de reserva para cada unidad funcional (ALU), donde se van acumulando las instrucciones.
- Nueva etapa de emisión (E): La última que se ejecuta en orden.
 - Se emite una instrucción cuando hay una estación de reserva disponible.
 - Los resultados de otras instrucciones se propagan a todas las estaciones de reserva a través del bus de datos común (CDB).
 - La instrucción espera en la estación de reserva hasta que conoce todos sus operandos.
 - En la espera puede ser adelantada por otras instrucciones posteriores.

18

- El algoritmo de Tomasulo:
 - Es un algoritmo distribuido para la planificación dinámica de instrucciones.
 - Para hacerlo posible:
 - Disponemos de varias unidades funcionales (ALUs de enteros, ALUs de coma flotante, etc.).
 - Cada unidad funciona dispone de una o varias estaciones de reserva.
 - Son lugares de espera para las instrucciones hasta que disponen de todos sus operandos.
 - En cada estación de reserva solo cabe una instrucción.
 - Se incluye una nueva etapa en el ciclo de ejecución denominada emisión (E).
 - Se emite una instrucción si y solo si hay una estación de reserva disponible para su operación.
 - Se asigna esta instrucción a esta estación de reserva durante el resto de ciclos hasta su finalización.
 - Como la emisión es una etapa en orden, si una instrucción no puede emitirse, debe esperar.
 - En la emisión, la instrucción realiza una copia local de sus operandos fuentes.
 - Esto equivale a renombrar los registros.
 - Esto elimina un tipo de riesgos de datos llamado *Write After Read* (WAR), en el que se produce una escritura después de una lectura.
 - Se evita el riesgo WAR porque cuando se produzca la escritura, ya se ha creado la copia local del registro en la estación de reserva.
 - Si en su emisión la instrucción dispone ya de todos sus operandos, puede pasar a la etapa de ejecución.
 - De lo contrario, permanece en la estación de reserva tantos ciclos como sean necesarios hasta que sus operandos son calculados por otras instrucciones.
 - Estos valores que está esperando la instrucción se comparten por el bus de datos común (CDB), que es donde cada instrucción escribe sus resultados.
 - Durante la espera, la instrucción puede ser adelantada por otras que sí disponen de sus operandos.
 - Aquí es donde surge la ejecución desordenada.
 - El bus de datos común (CDB) es un mecanismo similar al de los adelantamientos.
 - Permite también realizar un control de escrituras en el banco de registros que no introduzca paradas.

PLANIFICACIÓN DINÁMICA DE INSTRUCCIONES



19

- Algunas suposiciones en este diagrama:
 - Por un lado, las etapas IF e ID se realizan muy rápidamente (prebúsqueda de instrucciones).
 - Por otro, la etapa MEM se integra en EX.
 - Por eso no hemos dibujado estas etapas.
- Para este ejemplo, suponemos también que existe, entre otras:
 - Una ALU para enteros con 2 estaciones de reserva (no se usa en el ejemplo).
 - Una ALU para sumas en coma flotante con 2 estaciones de reserva.
 - Una ALU para multiplicación en coma flotante con 2 estaciones de reserva.
- La instrucción de multiplicación:
 - Puede emitirse en el primer ciclo porque está disponible la unidad de reserva 5.
 - Además dispone de sus operandos fuente (\$f2 y \$f4), con lo que pasa a ejecución en la ALU de multiplicaciones de coma flotante.
 - Suponemos que la multiplicación tarda 4 ciclos en completarse.
 - A continuación, escribe el resultado de la multiplicación en el bus CDB y en el registro \$f0 del banco.
- La primera instrucción de suma:
 - Puede emitirse en el segundo ciclo porque está disponible la unidad de reserva 3.
 - Sin embargo, se queda esperando en la estación durante 4 ciclos hasta que aparece el valor de \$f0 en el CDB.
 - Tras la espera puede pasar a ejecutarse en la ALU de sumas de coma flotante.
 - Suponemos que la suma tarda 2 ciclos en completarse.
 - Una vez terminada la suma, escribe el resultado de la suma en el bus CDB y en el registro \$f8.
- La segunda instrucción de suma:
 - Puede emitirse en el tercer ciclo porque está disponible la unidad de reserva 4.
 - Como ya dispone de sus operandos fuente (\$f4 y \$f6), pasa a ejecución en la ALU de sumas de coma flotante, que está disponible.
 - La suma tarda 2 ciclos en completarse.
 - Una vez terminada la suma, necesita incluir 1 ciclo de parada porque no pueden coincidir dos instrucciones en la misma etapa WB.
 - Eso significaría que estarían escribiendo a la vez en el bus CDB.
 - Transcurrida la parada, escribe el resultado en el bus CDB y en el registro \$f10.
- El resultado neto es que las instrucciones 1-2-3 han terminado su ejecución en el orden 1-3-2.
 - Se mantiene en orden la emisión.
 - El resto de etapas puede desordenarse.

RIESGOS DE CONTROL: SALTOS CONDICIONALES

- Decisión de salto condicional tomada en etapa MEM.
 - 3 ciclos de parada.



- Adelantar la decisión del salto a la etapa ID.
 - 1 ciclo de parada.

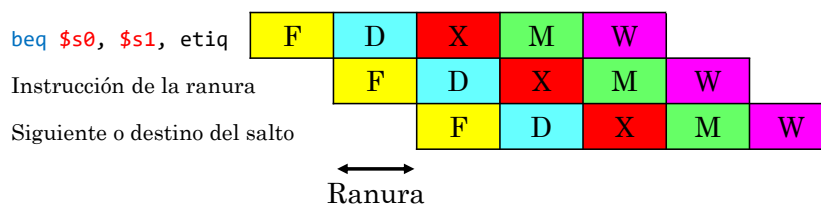


20

- En la primera versión del camino de datos segmentado de MIPS, la decisión de si se toma o no un salto condicional beq se produce en la etapa MEM.
 - Esto implica 3 ciclos de parada mientras se calcula cuál es la siguiente instrucción que hay que ejecutar:
 - La que va después de la instrucción del salto (cuando no se toma).
 - La del destino del salto (cuando sí se toma).
- Una primera mejora es modificar el camino de datos para adelantar la decisión a la etapa ID.
 - Para ello, necesitamos añadir un restador específico en dicha etapa para que, una vez leídos los dos registros, calcule la resta para ver si son iguales.
 - Por tanto, en esta etapa ID debe dar tiempo a hacer todo lo siguiente:
 - Proporcionar el código de operación a la unidad de control para decodificarla.
 - Leer dos registros en el banco, que son precisamente los implicados en la comparación de igualdad.
 - Restarlos para ver si son iguales y generar la señal de Cero.
 - Calcular también el destino del salto como la suma de la etiqueta que lleva la instrucción con el valor PC.
 - Actualizar el PC con el valor de esta suma solo si Cero = 1.
 - Esta modificación del camino de datos permite reducir a 1 ciclo la parada en caso de salto condicional.

RIESGOS DE CONTROL: SALTOS CONDICIONALES

- Ranura de retardo (o salto retardado, rellenado de ranura):
 - El compilador reordena el código para ejecutar una instrucción independiente durante el ciclo ID del salto (así se aprovecha el ciclo para hacer algo útil).
 - No siempre es posible.



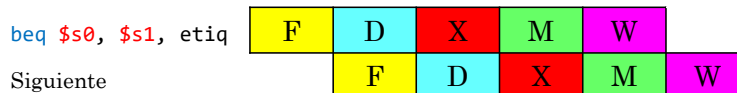
21

- La técnica de la ranura de retardo (o de salto retardado o rellenado de ranura):
 - Es una técnica de software para minimizar los riesgos de control.
 - La realiza el compilador cuando pasa el código fuente de alto nivel a ensamblador y código máquina.
 - Consiste en reordenar el código:
 - Para cada instrucción de salto condicional (como beq), intenta ejecutar justo después otra instrucción que no influya en el salto.
 - Suele ser una instrucción anterior al salto, aunque también se pueden elegir otras opciones.
 - De esta forma se aprovecha el ciclo ID, que sin esta técnica, sería obligatoriamente de parada.
 - Inmediatamente después de la instrucción de la ranura se ejecuta la instrucción correcta.
 - La que va después de la instrucción del salto (cuando no se toma).
 - La del destino del salto (cuando sí se toma).
 - No siempre se puede rellenar la ranura de retardo porque depende de que exista una instrucción que se pueda reubicar allí.

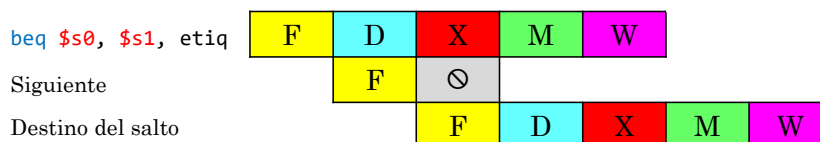
RIESGOS DE CONTROL: SALTOS CONDICIONALES

○ Predicción estática del salto:

- Se predice siempre que no habrá salto.
- Si la predicción es un acierto (no hay salto), todo OK.



- Si nos hemos equivocado (sí hay salto), se aborta la instrucción errónea y se lee la instrucción correcta.

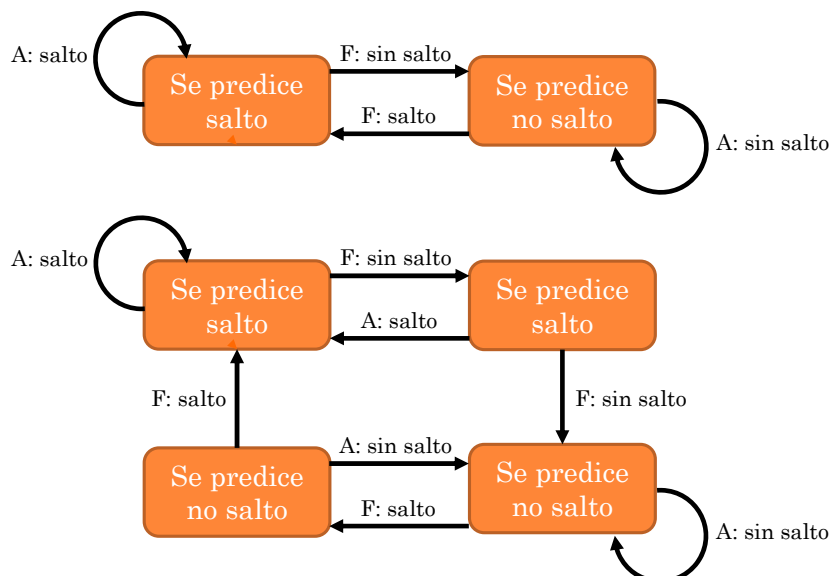


22

- Otra posibilidad para gestionar los riesgos de control es predecir siempre el mismo resultado para un salto condicional (predicción estática = fija).
 - En MIPS siempre se predice que no se tomará el salto.
 - Para otros procesadores podría ser interesante predecir que sí se tomará.
 - Como se predice que no habrá salto, durante el ciclo ID de la instrucción `beq` se comienza el ciclo IF de la instrucción que va justo después (dirección PC+4).
 - Al terminar el ciclo, se sabrá si hemos acertado o no.
 - Si hemos acertado, era correcto que no había salto, con lo que la instrucción que se ejecuta después de `beq` es la que viene después en el programa.
 - Por el contrario, si es un fallo, sí había que realizar el salto, con lo que hay que deshacer lo que hemos hecho mal.
 - La instrucción siguiente no continúa su ejecución, ya que en realidad no era la que había que ejecutar.
 - En el ciclo siguiente, a la vez que el ciclo EX de `beq`, comienza el ciclo IF de la instrucción destino del salto.
 - En la práctica, el caso de fallo equivale a 1 ciclo de parada, como antes.
 - Sin embargo, en caso de acierto ahorramos este ciclo.

RIESGOS DE CONTROL: SALTOS CONDICIONALES

○ Predicción dinámica de saltos:



23

- Realizar una predicción dinámica de saltos:
 - Permite decidir en tiempo de ejecución si conviene tomar los saltos o no.
 - La decisión se toma en función de los resultados de saltos previos.
 - La manera más sencilla es mediante una máquina de estados con 2 estados (figura superior).
 - Los estados son:
 - Se predice que hay salto.
 - Se predice que no hay salto.
 - Al ser 2 estados, la predicción se codifica con 1 bit.
 - Si comenzamos por el estado que predice salto:
 - Si acertamos (hay salto), seguimos en el mismo estado.
 - Si fallamos (no hay salto), pasamos al otro estado.
 - Si estamos en el estado que predice que no hay salto:
 - Si acertamos (no hay salto), seguimos en el mismo estado.
 - Si fallamos (hay salto), pasamos al otro estado.
 - Por tanto, con un acierto mantenemos el sentido de la predicción, mientras que con un fallo invertimos la predicción.
 - Este sistema es muy inestable.
 - Un único fallo aislado, nos hace cambiar la predicción, lo cual nos puede provocar más fallos a posteriori.
 - Para añadir estabilidad a la máquina de estados, la rediseñamos para que tenga ahora 4 estados (figura inferior).
 - Los estados son:
 - Dos de ellos predicen que habrá salto.
 - Los otros dos predicen que no habrá salto.
 - Al ser 4 estados, la predicción se codifica con 2 bits.
 - En esta nueva configuración, se necesitan dos fallos seguidos para cambiar el sentido de la predicción.
 - Para llevar la cuenta de lo que ocurre en cada salto, se necesitan estructuras de datos auxiliares que almacenen información relevante de cada posible salto.
 - Según el formato de esta estructura, se pueden almacenar todos o algunos de los siguientes datos:
 - La dirección de la instrucción de salto.
 - La predicción para ese salto (salto tomado/no tomado).
 - La dirección de la instrucción destino del salto.
 - Incluso la propia instrucción destino del salto.

PROCESADORES SUPERESCALARES

- En un **procesador superescalar** se permite la emisión simultánea de varias instrucciones.
 - Lectura de varias instrucciones en el mismo ciclo.
 - Se añaden más puertos de lectura y escritura en el banco de registros.
 - Se complica la unidad de control para detectar posibles dependencias entre instrucciones.
 - Se mantiene la emisión en orden de las instrucciones.
 - Se permite la ejecución de instrucciones mientras se calcula la decisión de un salto condicional.
 - Se ejecuta la instrucción predicha y se avanza en la ejecución hasta saber el resultado de la decisión.
 - A esta técnica se la llama **ejecución especulativa**.

24

- Hasta ahora se ha permitido el comienzo de la ejecución de una instrucción nueva por ciclo.
- Los procesadores superescalares están diseñados para permitir la emisión de dos o más instrucciones por ciclo.
 - Se trata de un valor máximo.
 - Si no hay unidades funcionales suficientes para la emisión simultánea, se emitirán solo aquellas instrucciones que puedan atenderse a la vez.
 - Quedará uno o varios huecos vacíos por cada instrucción que no se haya podido emitir, con lo que no se aprovechará al máximo las capacidades superescalares del procesador.
 - Para poder emitir varias instrucciones en el mismo ciclo:
 - Se modifica la memoria para poder leer varias instrucciones a la vez.
 - Se añaden más puertos (accesos) al banco de registros para permitir varias lecturas y escrituras.
 - Hay que rediseñar la unidad de control, que debe detectar más dependencias entre instrucciones.
 - No se puede emitir en el mismo ciclo una instrucción que depende de otra del mismo ciclo.
 - En cuanto al orden de las diferentes etapas de ejecución:
 - Se mantiene la emisión en orden de las instrucciones (después de IF e ID, que siguen siendo muy rápidas).
 - Se permite la emisión de una instrucción, aunque dependa de la realización o no de un salto condicional.
 - Es decir, se predice un resultado para el salto y se avanza en la ejecución de instrucciones hasta que se confirma.
 - A esta técnica se la llama ejecución especulativa.
 - Las etapas de ejecución pueden desordenarse, según las unidades funcionales disponibles.
 - (Continúa en la siguiente diapositiva).

PROCESADORES SUPERESCALARES

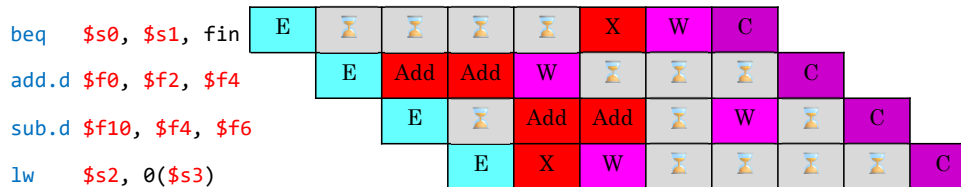
- Se añade una nueva etapa de finalización C (*Commit*), que debe realizarse en orden.
 - Se produce cuando se resuelven todas las decisiones de salto que estaban pendientes.
 - De esta forma, se confirman todos los cálculos realizados hasta el momento.
 - Si la predicción del salto fue errónea, no se produce el *commit* y se desechan los cálculos.
- Emisión y *commit* se producen en orden, pero la ejecución es fuera de orden.
- En la etapa WB se escriben los resultados en el bus CDB, mientras que en C se confirman los resultados al escribirlos en el banco de registros.

25

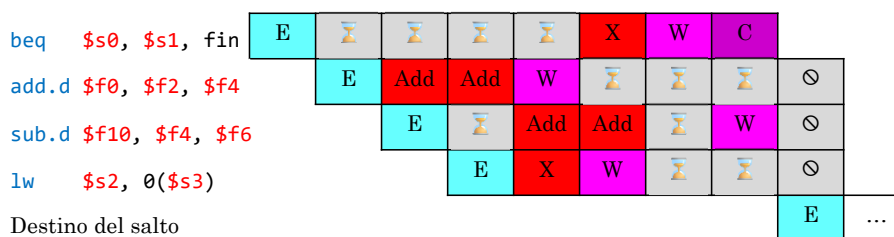
- En cuanto al orden de las diferentes etapas de ejecución:
 - Se añade una nueva etapa llamada *Commit* (C).
 - Es siempre la última etapa de cada instrucción.
 - Debe producirse en orden, aunque las instrucciones se hayan ejecutado en desorden.
 - En esta etapa se confirman todos los resultados intermedios de las operaciones realizadas y se guardan en los registros correspondientes.
 - Si hay instrucciones que dependían de un salto, que se demuestra que no se toma, sus resultados son descartados y las instrucciones son expulsadas del camino de datos.
 - La escritura de los resultados se produce:
 - En el bus de datos común CDB durante la etapa WB.
 - Esto permite que otras instrucciones que necesitan estos resultados puedan avanzar en su ejecución.
 - En el banco de registros durante la etapa C, cuando se confirma que no dependen de ningún salto anterior.
 - Es cuando los resultados se hacen definitivos.

PROCESADORES SUPERESCALARES

- En el *commit* de **beq** se confirma el acierto de la predicción del salto:



- En el *commit* de **beq** se rechaza la predicción:



26

- Por simplicidad, consideraremos en este ejemplo la emisión de 1 sola instrucción por ciclo.
- Supongamos que tenemos una instrucción de salto condicional **beq** que debe esperar 4 ciclos hasta conseguir sus operandos (\$s0 y \$s1).
- Mediante un predictor de salto, se predice cuál será la siguiente instrucción.
 - Supongamos que la predicción dice que no habrá salto.
- Sin esperar a confirmar la corrección de la predicción, se sigue ejecutando el programa, como si la predicción fuera correcta.
 - Se ejecutan las instrucciones que están inmediatamente después del **beq**.
- Los resultados se escriben en el bus CDB (etapa W), pero la escritura definitiva en el banco de registros se retrasa hasta que llegue el *commit* de la instrucción **beq**.
 - Si se confirma el acierto en la predicción, todos los resultados intermedios se hacen definitivos mediante las etapas de *commit* de las instrucciones.
 - Estos *commits* se realizan en orden, a pesar de que la ejecución ha podido ser desordenada (como en el ejemplo).
 - Si se prueba que la predicción fue errónea, todos los resultados intermedios calculados desde **beq** son descartados.
 - No se realiza el *commit* de dichas instrucciones.
 - Debemos empezar desde el principio la emisión de la instrucción correcta después de **beq**.

CONTENIDOS

- Introducción
- Paralelismo a nivel de instrucción
 - Segmentación de instrucciones
 - Riesgos de datos RAW
 - Adelantamientos
 - Procesadores multifuncionales
 - Riesgos de datos WAW
 - Planificación dinámica de instrucciones
 - Riesgos de control (saltos condicionales)
 - Procesadores superescalares
- Paralelismo a nivel de hilo
- Paralelismo a nivel de procesador
 - Procesadores multinúcleo
 - Multiprocesadores simétricos
 - Acceso no uniforme a memoria
 - *Clusters*
- Paralelismo de datos
 - Procesadores vectoriales
 - Unidad de procesamiento gráfico (GPU)
- Despedida

PROCESADORES CON MULTITHILO

- Los **procesadores con multihilo** (*multithreading*) permiten la ejecución de dos o más hilos pertenecientes al mismo programa.
 - Es como tener varios procesadores virtuales o lógicos (pero en realidad es solo uno).
- Cada hilo cuenta con su propia pila y contador de programa, pero comparte código y variables con el resto de hilos.
- Tipos:
 - **Multihilo simétrico** (SMT): se emiten a la vez varias instrucciones de hilos diferentes.
 - **Multihilo de grano grueso** (CMT): se emiten a la vez varias instrucciones del mismo hilo (o de un hilo o de otro).

28

- A pesar de que un procesador superescalar puede emitir varias instrucciones en el mismo ciclo, no siempre se consigue porque las unidades funcionales son limitadas.
- La solución es usar un procesador con *multithreading* (multihilo):
 - Un hilo de ejecución es la secuencia de instrucciones más pequeña que puede manejar un planificador dinámico.
 - Un procesador con *multithreading* puede ejecutar a la vez varios hilos del mismo programa.
 - Es como tener dos o más procesadores virtuales o lógicos, cada uno ejecutando un hilo.
 - Cada hilo:
 - Cuenta con su propia pila y su propio contador de programa.
 - Comparte con el resto de los hilos el código del programa, así como las variables.
 - Según cómo esté implementado el *multithreading*, hay dos tipos:
 - Simétrico (*Symmetrical multithreading*, SMT, también llamado Multihilo Simultáneo, *Simultaneous multithreading*):
 - En cada ciclo se emiten varias instrucciones de hilos diferentes.
 - Para ello se necesita poder leer instrucciones de diferentes hilos en el mismo ciclo.
 - También se hace necesario un banco de registros más grande para albergar datos de los diferentes hilos.
 - De grano grueso (*Coarse-grained multithreading*, CMT, también llamado Multihilo Temporal, *temporal multithreading*):
 - En cada ciclo se emiten varias instrucciones del mismo hilo.
 - Los hilos se van alternando, según los recursos disponibles y las necesidades de las instrucciones, con cambios de hilo al comienzo de determinados ciclos.
 - Para ello los cambios de hilo deben ser especialmente rápidos.

CONTENIDOS

- Introducción
- Paralelismo a nivel de instrucción
 - Segmentación de instrucciones
 - Riesgos de datos RAW
 - Adelantamientos
 - Procesadores multifuncionales
 - Riesgos de datos WAW
 - Planificación dinámica de instrucciones
 - Riesgos de control (saltos condicionales)
 - Procesadores superescalares
- Paralelismo a nivel de hilo
- Paralelismo a nivel de procesador
 - Procesadores multinúcleo
 - Multiprocesadores simétricos
 - Acceso no uniforme a memoria
 - *Clusters*
- Paralelismo de datos
 - Procesadores vectoriales
 - Unidad de procesamiento gráfico (GPU)
- Despedida

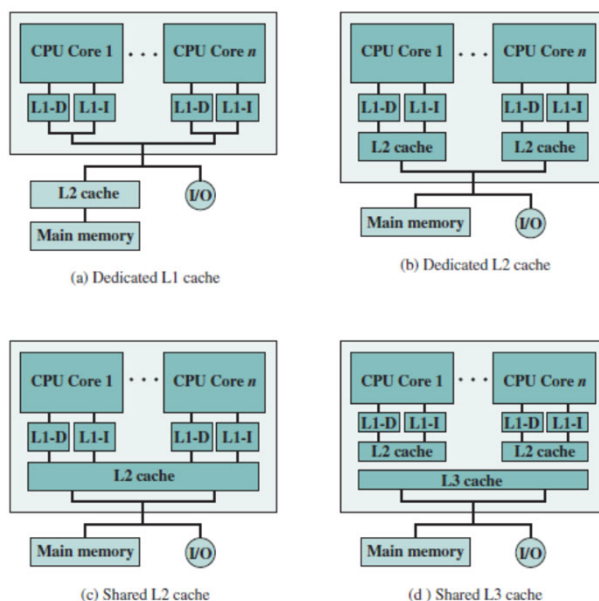
PROCESADORES MULTINÚCLEO

- Los **procesadores multinúcleo** contienen varios procesadores completos en el mismo chip, llamados ahora **núcleos** (*cores*).
 - Cada núcleo contiene todos los elementos habituales de un procesador:
 - Registros, ALU, camino de datos, unidad de control, caché, etc.
- Factores de diseño:
 - Núcleos por chip.
 - Niveles de caché.
 - Exclusivos a cada núcleo/compartidos entre núcleos.
 - Unificada/separada para datos e instrucciones.

30

- Hasta ahora hemos considerado que el chip contiene un único procesador o CPU.
- Los procesadores multinúcleo (*multicore*) disponen de varios núcleos en el mismo chip.
 - Cada núcleo (*core*) es una CPU completa que puede ejecutar sus propios programas.
 - A diferencia del caso multihilo, ahora se permite la ejecución simultánea de instrucciones de diferentes programas.
 - Puede haber recursos propios a cada núcleos o compartidos entre varios núcleos.
- Factores de diseño:
 - El número de núcleos contenidos en el chip.
 - El número de niveles (*levels*) de la memoria caché.
 - Exclusivos a un núcleo o compartidos entre varios núcleos.
 - Unificada para datos e instrucciones o separadas para datos y para instrucciones.

PROCESADORES MULTINÚCLEO



31

- Diferentes configuraciones para procesadores multinúcleo:
 - a) Cachés L1 separadas de datos y de instrucciones para cada núcleo, más una caché L2 unificada para datos e instrucciones y compartida para todos los núcleos, pero ubicada en un chip externo al procesador.
 - b) Cachés L1 separadas de datos y de instrucciones para cada núcleo, más una caché L2 unificada para datos e instrucciones, pero de uso exclusivo para el núcleo, todo en el mismo chip.
 - c) Cachés L1 separadas de datos y de instrucciones para cada núcleo, más una caché L2 unificada para datos e instrucciones y compartida para todos los núcleos, todo en el mismo chip.
 - d) Cachés L1 separadas de datos y de instrucciones para cada núcleo, más una caché L2 unificada para datos e instrucciones y exclusiva a cada núcleo, más una caché L3 unificada para datos e instrucciones y compartida para todos los núcleos, todo en el mismo chip.

Imagen tomada de: William Stallings, Computer Organization and Architecture, 9.^a edición, 2013, pág. 675.

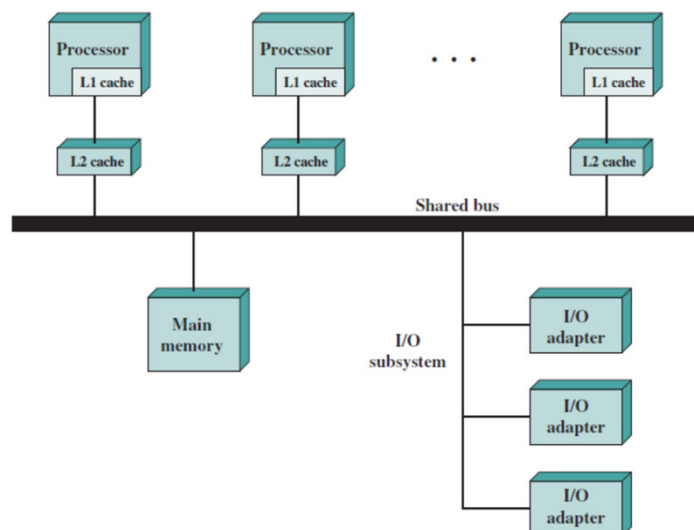
MULTIPROCESADORES SIMÉTRICOS

- Los **multiprocesadores simétricos** (SMP) tienen las siguientes características:
 - Hay dos o más procesadores similares de capacidades comparables, que comparten la memoria principal y las E/S y que están interconectados mediante un bus o un sistema de interconexión equivalente.
 - El tiempo de acceso a la memoria es aproximadamente el mismo para todos los procesadores.
 - Todos los procesadores pueden desempeñar las mismas funciones.
 - El sistema está controlado por un sistema operativo integrado que proporciona la interacción entre los procesadores y sus programas a los niveles de trabajo, tarea, fichero y datos.

32

- Conviene no confundir los conceptos de multiprogramación y multiprocesamiento:
 - Multiprogramación:
 - Hace referencia a la ejecución simultánea de varios programas.
 - En realidad, en un momento dado el procesador está ejecutando un único programa.
 - Lo que pasa es que, cada cierto tiempo, y de manera muy rápida, va saltando de programa en programa, repartiendo por tanto el tiempo entre todos los que están en ejecución.
 - A efectos prácticos, el usuario ve que hay varios programas ejecutándose a la vez.
 - Multiprocesamiento:
 - Se refiere a que hay varios procesadores, cada uno de los cuales puede estar ejecutando un programa diferente.
 - Por lo tanto, en un momento dado, puede haber dos o más programas ejecutándose en el computador.
- Los multiprocesadores simétricos (SMP, *symmetric multiprocessors*) muestran grandes beneficios:
 - Prestaciones:
 - Si el trabajo se puede organizar para ejecutarse en paralelo, disponer de varios procesadores obviamente proporcionará mejores prestaciones que tener uno solo.
 - Disponibilidad:
 - En caso de fallo, el trabajo lo podrá realizar otro de los procesadores, con lo que el computador no se bloquea.
 - Crecimiento incremental y escalado:
 - Se pueden aumentar las prestaciones del sistema añadiendo más procesadores.
 - Oferta comercial:
 - A nivel comercial, se ofrecen productos de diferentes precios y prestaciones según el número de procesadores del sistema.
- En un SMP, la existencia de múltiples procesadores es transparente al usuario.
 - Es el sistema operativo el encargado de sincronizar los procesadores y de planificar los hilos y asignarlos a los distintos procesadores.

MULTIPROCESADORES SIMÉTRICOS



33

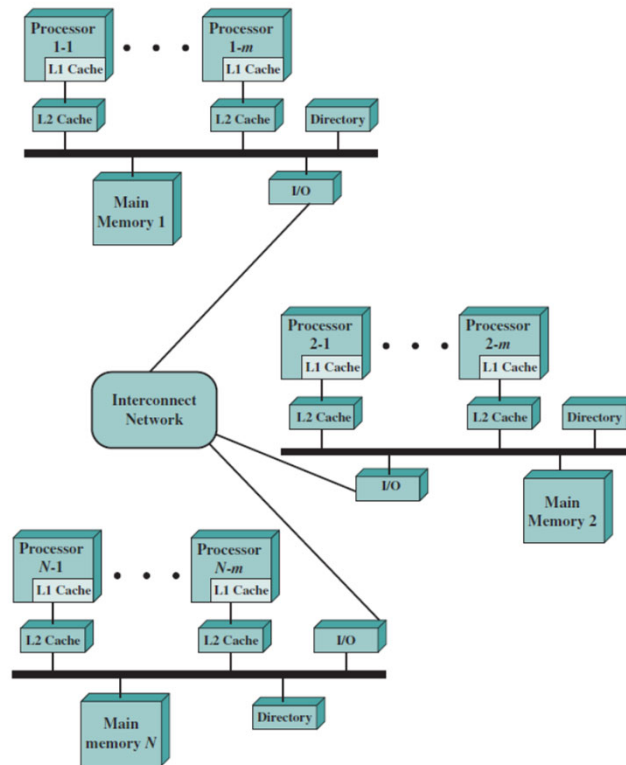
- La organización más común de un multiprocesador simétrico es la que utiliza un bus de tiempo compartido.
 - La estructura y las interfaces son básicamente las mismas que las de un sistema monoprocesador basado en un bus.
 - Ahora tenemos varios procesadores, cada uno con sus propias cachés (puede haber también algún nivel de caché compartido entre los procesadores), además de la memoria principal y los adaptadores de E/S.
 - Como es habitual, el bus dispone de líneas de datos, de control y de dirección.
 - Como es un recurso compartido, debe haber algún protocolo de arbitraje para el uso del bus.
 - De esta forma, si un componente está controlando el bus, los demás no tienen acceso al mismo y deben, si es necesario, suspender su operación hasta que dispongan del bus.
 - Esta organización basada en un bus compartido es interesante por su simplicidad, flexibilidad y fiabilidad.
 - Sin embargo, también presenta algún inconveniente:
 - El bus se convierte en un cuello de botella.
 - La introducción de memorias caché en cada procesador reduce drásticamente el uso del bus.
 - Lamentablemente, puesto que cada caché local contiene una imagen de una parte de la memoria, si se altera una palabra en una caché, podrían quedar copias anticuadas en otras cachés.
 - Este problema se denomina **coherencia de caché**.
 - Puede resolverse mediante diversos mecanismos hardware:
 - Protocolo de directorio:
 - Un controlador detecta qué cambios se han hecho en una caché y avisa a las otras cachés para que actualicen el bloque de datos, o bien, lo marquen como no válido.
 - Protocolo de sondeo (*snoopy*):
 - Cada caché tiene un controlador que espía los cambios que hacen las otras cachés, ya que estas publican los cambios mediante un sistema de difusión (*broadcast*).
 - También existen mecanismos software:
 - El compilador se encarga de detectar problemas potenciales con los datos y los marca como "no cacheables", es decir, como datos que nunca pueden llevarse a la caché.

ACCESO NO UNIFORME A MEMORIA

- **Acceso no uniforme a memoria (NUMA):** el tiempo de acceso depende de la región a la que se acceda. Hay memoria “local” y memoria “remota”, aunque desde el punto de vista lógico, los procesadores ven un único espacio de memoria.
- **NUMA con coherencia de caché (CC-NUMA):** un tipo especial de NUMA en el que la coherencia de caché se mantiene en todas las cachés de los distintos procesadores.
- **Arquitectura de solo caché (COMA):** las memorias principales de cada nodo se usan como caché del resto del sistema.

34

- Los SMP proporcionan acceso uniforme a memoria (UMA, *Uniform Memory Access*), ya que el tiempo de acceso de un procesador a cualquier región de la memoria es el mismo.
- En los sistemas con acceso no uniforme a memoria (NUMA, *Non-uniform Memory Access*):
 - El tiempo de acceso no es constante, porque existe una memoria:
 - Local (más cercana al procesador en cuestión).
 - Remota (ubicada en los otros procesadores y, por tanto, lejos).
 - Si incluyen mecanismos para garantizar la coherencia de caché, se denominan CC-NUMA:
 - Ventajas:
 - Alcanzan mayor nivel de paralelismo que los SMP, sin cambios importantes en el software.
 - El tráfico del bus en cualquier nodo se limita a las peticiones que el bus puede manejar.
 - Las cachés locales de cada nodo reducen los accesos a memoria, incluso los remotos.
 - Inconvenientes:
 - No parecen un SMP de manera transparente.
 - El sistema operativo de un SMP necesita adaptación (asignación de páginas virtuales y de procesos y equilibrado de carga).
 - También los programas necesitan adaptarse.
 - La disponibilidad puede ser un problema.



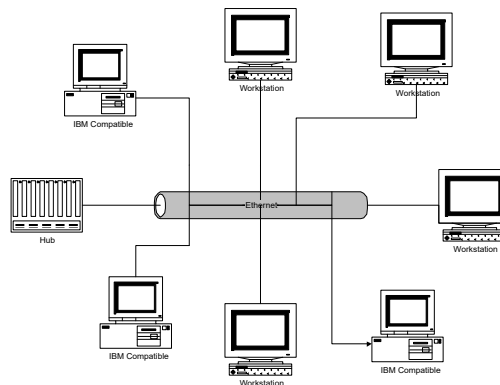
35

- Configuración típica de un computador CC-NUMA.
 - En la imagen se muestran varios nodos independientes, cada uno de los cuales es, en este caso, un SMP (en cada nodo hay varios procesadores, cada uno con dos niveles de caché, más la memoria principal).
 - El nodo es el bloque básico de construcción de toda la organización CC-NUMA.
 - Los nodos están interconectados mediante un medio de comunicación, que podría ser algún mecanismo de conmutación, un anillo o algún tipo de red.
 - Cada nodo incluye cierta cantidad de memoria principal.
 - Sin embargo, desde el punto de vista de los procesadores, existe un único espacio de memoria direccionable en el que a cada posición se asocia una única dirección válida para todo el sistema.
 - Cuando un procesador inicia un acceso a memoria, si la posición solicitada no se encuentra en la caché del procesador, entonces la caché L2 inicia una operación de captación.
 - Si la línea deseada está en una posición remota de la memoria, entonces se envía una petición automática para captar dicha línea a través de la red de interconexión, se proporciona a través del bus local a la caché que lo solicitaba en dicho bus.
 - Toda esta actividad es automática y transparente al procesador y a su caché.
 - En cuanto a la coherencia de caché, cada nodo debe mantener algún tipo de orden de directorios que dé una indicación de la situación de varias partes de la memoria y también de la información de estado de la caché.

Imagen tomada de: William Stallings, Computer Organization and Architecture, 9.ª edición, 2013, pág. 642.

CLUSTERS

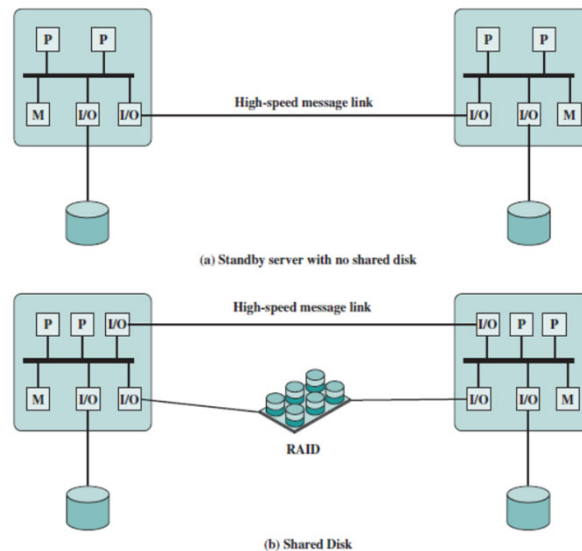
- Un *cluster* es un grupo de ordenadores completos, independientes e interconectados que trabajan conjuntamente como un único recurso de cómputo (como si fueran una única máquina).



36

- Con la expresión “ordenador completo” nos referimos a que cada uno de los ordenadores que forman el *cluster* (nodos) puede funcionar por sí solo, independientemente del *cluster*.
 - Pueden ser ordenadores muy normalitos, que, con sinergia, pueden producir grandes resultados si trabajan de manera colaborativa.
- Algunos de los beneficios del uso de *clusters* son los siguientes:
 - Escalabilidad absoluta:
 - Es posible configurar *clusters* grandes que incluso superen las prestaciones de los computadores independientes más potentes.
 - Escalabilidad incremental:
 - El *cluster* es fácilmente ampliable mediante la incorporación de nuevos nodos.
 - Alta disponibilidad:
 - Como cada nodo es independiente, el fallo de uno de los nodos no significa la pérdida del servicio.
 - Mejor relación precio para las mismas prestaciones:
 - Como utilizan elementos comerciales de bajo precio, se consiguen altas prestaciones a un precio reducido.
- Existe un tipo muy extendido de *cluster* llamado “Beowulf”.
 - Consiste en conectar computadores idénticos, de gama media-baja, conectados mediante una red de área local, normalmente bajo un sistema operativo Linux o similar.
 - Aunque individualmente sean computadores de prestaciones modestas, como *cluster* pueden llegar a tener altas prestaciones.

CLUSTERS



37

- Arriba:
 - *Cluster* configurado como nodos independientes sin disco compartido.
- Abajo:
 - *Cluster* configurado como nodos que comparten un disco basado en tecnología RAID (son varios discos duros en los que se guarda la información de manera redundante, de tal forma que, en caso de fallo, se puede recuperar la información contenida en el disco que da el fallo).
- En ambos casos, cada nodo del *cluster* es un computador multiprocesador (P) con su propia memoria principal (M) y su disco duro.
- Además, se ha dibujado una conexión específica de alta velocidad para el intercambio de mensajes entre los nodos del *cluster*.

Imagen tomada de: William Stallings, Computer Organization and Architecture, 9.ª edición, 2013, pág. 634.

CONTENIDOS

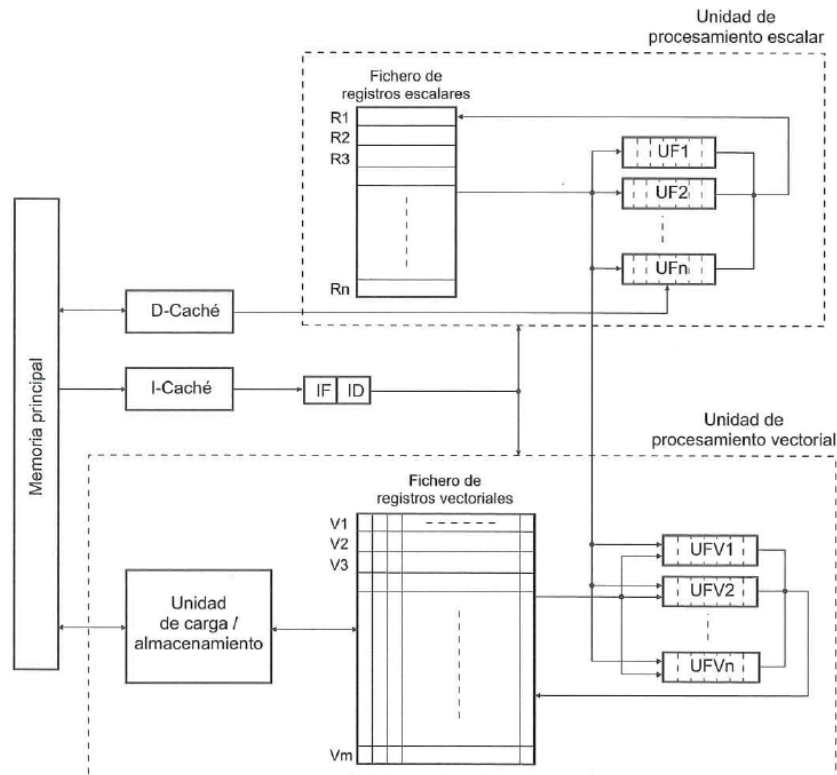
- Introducción
- Paralelismo a nivel de instrucción
 - Segmentación de instrucciones
 - Riesgos de datos RAW
 - Adelantamientos
 - Procesadores multifuncionales
 - Riesgos de datos WAW
 - Planificación dinámica de instrucciones
 - Riesgos de control (saltos condicionales)
 - Procesadores superescalares
- Paralelismo a nivel de hilo
- Paralelismo a nivel de procesador
 - Procesadores multinúcleo
 - Multiprocesadores simétricos
 - Acceso no uniforme a memoria
 - *Clusters*
- Paralelismo de datos
 - Procesadores vectoriales
 - Unidad de procesamiento gráfico (GPU)
- Despedida

PARALELISMO DE DATOS

- Los procesadores vectoriales proporcionan instrucciones que trabajan sobre vectores de datos.
 - Una instrucción vectorial equivale a un bucle.
 - También tienen instrucciones para trabajar con datos escalares.
- Registros en un procesador vectorial: escalares/vectoriales.
- Unidades funcionales en un procesador vectorial:
 - Registros vectoriales: para almacenar vectores completos.
 - Unidades funcionales vectoriales: están completamente segmentadas.
 - Unidad de carga / almacenamiento de vectores.
 - Registros escalares: para almacenar datos simples.
 - Unidades funcionales escalares.
- Ventajas:
 - Vectores: tienen un patrón de acceso a memoria conocido.
 - Se reducen los riesgos de control.

39

- Pasemos ahora a mencionar ejemplos de paralelismo de datos (SIMD).
 - Los procesadores vectoriales son de tipo SIMD y son de propósito específico, ya que están especialmente diseñados para trabajar con vectores o matrices.
 - En estos procesadores, se accede de manera natural y directa a las componentes de un vector, de tal forma que una instrucción vectorial (como, por ejemplo, la suma de dos vectores), realmente se contempla como una única operación.
 - Es por ello que la instrucción vectorial equivale a un bucle.
 - Para poder realizar esto, disponen de bancos de registros vectoriales, de unidades funcionales vectoriales (unidades funcionales repetidas que actúa cada una sobre una de las componentes del vector), etc.
 - Obviamente también disponen de instrucciones y de unidades funcionales específicas para datos escalares.



- Esta imagen muestra un posible esquema para un procesador vectorial.
 - Dispone de una unidad de procesamiento escalar:
 - Como en cualquier procesador normal.
 - Dispone de su banco de registros, así como de varias unidades funcionales (ALUs) para trabajar con dichos datos escalares.
 - Pero también dispone de una unidad de procesamiento vectorial:
 - Está adaptada para trabajar con múltiples componentes de un vector.
 - Por ello el banco de registros almacena varios datos diferentes por registro.
 - Las unidades funcionales (ALUs) permiten realizar las operaciones de manera vectorial, es decir, simultánea para varias componentes.
 - La unidad de carga/almacenamiento es la encargada de leer múltiples componentes de un vector de la memoria (carga) o escribirlas en la memoria (almacenamiento).
 - De esta forma, si quisiéramos sumar dos vectores de coma flotante de 32 componentes:
 - La operación `lw` vectorial cargaría las 32 componentes del primer vector en uno de los registros.
 - Una segunda operación `lw` vectorial haría lo propio con el segundo vector.
 - A continuación, ambos datos se llevarían ambos vectores a la unidad sumadora-restadora de coma flotante, de tal forma que en un solo ciclo de reloj se sumarían componente a componente.
 - El resultado podría almacenarse en otro de los registros vectoriales del banco.

UNIDAD DE PROCESAMIENTO GRÁFICO (GPU)

- Una **GPU** es un coprocesador específico para el procesamiento gráfico y las operaciones de coma flotante.
 - Altamente segmentado.
 - Múltiples unidades funcionales, basadas en píxeles y en vértices.
 - Con miles de núcleos muy sencillos.
 - Con memoria muy rápida.
- Permiten aligerar el trabajo de la CPU, que se dedica a otros cálculos.
- **GPGPU**: programación de propósito general con GPUs.

41

- La tarjeta gráfica contiene el chip específico que crea y procesa las imágenes que se muestran por pantalla.
 - Debido a su creciente complejidad, desde 1999 se llaman GPU (unidad de procesamiento gráfico, *graphics processing unit*), cuando Nvidia presentó su GeForce 256.
 - Hoy en día las GPUs son de gran ayuda en gráficos por computador y procesamiento de imagen.
- Características de una GPU:
 - Muchas etapas de segmentación.
 - Muchas unidades funcionales.
 - Unas trabajan con píxeles y otras lo hacen con vértices.
 - Muchos núcleos (miles).
 - Acceso muy rápido a la memoria.
- Cuando estos cálculos de los gráficos los hace la GPU, la CPU se puede dedicar a ejecutar otras instrucciones, con lo que se avanza en la ejecución del programa.
- También ha surgido la programación de propósito general aprovechando las características de las GPUs.
 - Es lo que se denomina GPGPU (*General-purpose computing on graphics processing units*).

CONTENIDOS

- Introducción
- Paralelismo a nivel de instrucción
 - Segmentación de instrucciones
 - Riesgos de datos RAW
 - Adelantamientos
 - Procesadores multifuncionales
 - Riesgos de datos WAW
 - Planificación dinámica de instrucciones
 - Riesgos de control (saltos condicionales)
 - Procesadores superescalares
- Paralelismo a nivel de hilo
- Paralelismo a nivel de procesador
 - Procesadores multinúcleo
 - Multiprocesadores simétricos
 - Acceso no uniforme a memoria
 - *Clusters*
- Paralelismo de datos
 - Procesadores vectoriales
 - Unidad de procesamiento gráfico (GPU)
- Despedida

DESPEDIDA

- En la actualidad, la inmensa mayoría de los procesadores son **ARM** (procesadores RISC).
 - Móviles y tabletas.
 - Más de 310 mil millones de chips vendidos hasta la fecha (*).
- Procesadores x86 (CISC) – sobremesa y portátiles (**):
 - Sobremesa: Intel (66,4 %) – AMD (33,6 %).
 - Portátiles: Intel (78,1 %) – AMD (21,9 %).
- Comparación global x86/ARM (***)
 - Intel+AMD x86 (88,4 %).
 - ARM (11,6 %).

(*) Datos del primer trimestre de 2025: <https://investors.arm.com/static-files/9be77c9d-75ee-4639-bfe4-17efd23c56b5>

(**) Datos del tercer trimestre de 2025: <https://www.tomshardware.com/pc-components/cpus/amd-continues-to-chip-away-at-intels-x86-market-share-company-now-sells-over-25-percent-of-all-x86-chips-and-powers-33-percent-of-all-desktop-systems>

(***) Datos del tercer trimestre de 2025: https://www.theregister.com/2025/11/14/mercury_amd_intel/

43

- En este curso hemos estudiado a fondo la arquitectura del procesador MIPS.
 - Se trata de un procesador de repertorio de instrucciones reducido (RISC, *reduced instruction set computer*).
 - Pocas instrucciones en el repertorio.
 - Pocos modos de direccionamiento.
 - Cada instrucción realiza una operación muy básica.
 - Instrucciones de tamaño regular para facilitar su decodificación.
 - Programas más largos.
 - El diseño del camino de datos es sencillo y regular.
 - Hoy en día, el mercado de los procesadores está copado por una familia de procesadores llamados ARM, que también son RISC.
 - Usados en móviles y tabletas.
 - Parecidos a MIPS.
 - Hasta el 31 de marzo de 2025, se han vendido más de 310 mil millones de chips basados en ARM.
 - Los chips ARM están empezando a entrar en el mercado de los ordenadores gracias a Apple.
 - En el tercer trimestre de 2025, los procesadores ARM han arañado un 11,6 % de todo el mercado de los chips, un valor que se va incrementando lentamente en los últimos años.
- En cuanto a Intel:
 - Fue la empresa que inventó el primer procesador en un único chip (1971).
 - Desde entonces domina el mercado de los chips para ordenadores y servidores.
 - Estos procesadores se clasifican como CISC (repertorio de instrucciones complejo, *complex instruction set computer*).
 - Muchas instrucciones.
 - Instrucciones complicadas, que hacen muchas operaciones en la misma instrucción.
 - Muchos modos de direccionamiento.
 - Instrucciones de longitud variable, lo que complica su decodificación.
 - Los chips de Intel forman la gran familia de procesadores x86.
 - Tiene un gran competidor: AMD.
 - Según datos conocidos del tercer trimestre de 2025, el 66,4 % de los chips x86 para ordenadores de sobremesa son de Intel y el resto (33,6 %) de AMD.
 - En cuanto a los datos para portátiles, Intel resiste con un 78,1 % frente a un 21,9 % para AMD.
 - La tendencia que se observa en los últimos años es:
 - Un aumento de AMD en detrimento de Intel solamente en cuanto a los procesadores x86.
 - Un ligero aumento de ARM (por el tirón de Apple) en detrimento de los procesadores x86 de Intel y AMD.

Our World
in Data

The graph illustrates the exponential growth of transistor counts in integrated circuits over time, following Moore's Law. The y-axis represents the transistor count on a logarithmic scale, ranging from 1,000 to 50,000,000,000. The x-axis represents the year, from 1970 to 2020. The data points show a consistent upward trend, with transistor counts doubling approximately every two years. Key milestones include the Intel 4004 (1971), Intel 8086 (1982), Intel Pentium (1992), Intel Core 2 Duo (2006), and various ARM and AMD processors in the 2010s and 2020s. The plot demonstrates the rapid pace of technological advancement in semiconductor manufacturing.

Licensed under [CC-BY](#) by the author Max Roser.

44

- 44